Promptel: A Declarative Framework for Advanced Prompt Engineering with Harmony Protocol Integration

Dipankar Sarkar
dipankar@terraprompt.com

September 30, 2025

Abstract

Enterprise deployment of large language models faces systematic engineering challenges: prompts embedded in code lack version control semantics, advanced reasoning techniques require manual implementation for each use case, and regulatory compliance demands auditability that string templates cannot provide. This paper introduces Promptel, a domain-specific language (DSL) for declarative prompt engineering that treats prompts as first-class artifacts with formal syntax, type systems, and execution semantics. The framework provides dual-format support (.prompt and YAML), native Harmony Protocol integration for multi-channel reasoning, and provider-agnostic execution across OpenAI, Anthropic, and Groq APIs. We present Promptel's formal grammar, denotational semantics for AST evaluation, and technique transformation algorithms. A migration study with 15 production prompts in financial services demonstrates quantifiable improvements: 45% reduction in lines of code, elimination of runtime type errors through compile-time validation, 85% technique reuse through compositional semantics, and full audit trail compliance through metadata tracking and channel separation. The framework addresses critical gaps in regulated AI deployment where traditional ad-hoc approaches fail to meet compliance, maintainability, and systematic engineering requirements. This work contributes formal foundations for prompt engineering as a software engineering discipline, with implications for enterprise LLM application development, regulatory compliance tooling, and domain-specific language design for AI systems.

1 Introduction

The integration of large language models into enterprise software systems represents a fundamental shift in how organizations approach knowledge work automation [1, 2]. However, the transition from experimental deployments to production-grade systems has revealed significant engineering challenges, particularly in regulated industries where determinism, auditability, and compliance are non-negotiable requirements [3].

Financial services institutions face unique constraints when deploying LLM-based systems. Regulatory frameworks such as MiFID II in Europe and SEC regulations in the United States mandate explainability, auditability, and human oversight of automated decision-making systems [4]. Traditional approaches to prompt engineering—embedding natural language instructions directly in application code—fail to meet these requirements. They lack version control semantics, offer no formal structure for technique composition, and provide no mechanism for systematic validation or testing.

This paper presents Promptel, a declarative framework that addresses these gaps through three primary contributions:

(1) Dual-format declarative specification: A domain-specific language supporting both native .prompt syntax and YAML serialization, enabling human-readable prompt definitions with full version control integration and configuration management compatibility.

(2) Native Harmony Protocol integration: The first Node.js framework to support OpenAI's Harmony Protocol [5], enabling multi-channel reasoning with separated analysis, commentary, and final output streams—critical for regulatory compliance and model interpretability.

(3) Provider-agnostic architecture: An abstraction layer supporting OpenAI, Anthropic, and Groq APIs while preserving provider-specific capabilities such as thinking tokens and reasoning effort parameters.

The framework's design philosophy centers on treating prompts as first-class artifacts in the software development lifecycle, analogous to SQL queries in database systems or HTML templates in web frameworks. This approach enables systematic testing, version control, and collaborative development of prompt-based systems.

We motivate our design through examination of financial services use cases, including regulatory document analysis, risk assessment automation, and customer communication systems. These domains exemplify the challenges Promptel addresses: complex multi-step reasoning requirements, stringent auditability demands, and the need for deterministic behavior within probabilistic systems.

The remainder of this paper is organized as follows: Section 2 presents a motivating example demonstrating current prompt engineering challenges. Section 3 reviews related work. Section 4 presents Promptel's DSL design. Section 5 details transformation and execution semantics. Section 6 provides comparative evaluation. Section 7 examines financial services applications. Section 8 discusses limitations and future work. Section 9 concludes.

2 Motivating Example: The Prompt Engineering Crisis

2.1 Current State of Practice

Consider a financial services firm deploying an LLM-based regulatory compliance analysis system. The initial implementation embeds prompts directly in Python code:

Listing 1: Ad-hoc prompt engineering (current practice)

```
def analyze_regulation(reg_text, portfolio, jurisdiction):
    prompt = f"""Analyze␣regulatory␣change␣impact:

Regulation:␣{reg_text}
Portfolio:␣{portfolio}
Jurisdiction:␣{jurisdiction}

Think␣step␣by␣step:
1.␣Parse␣the␣regulation
2.␣Identify␣affected␣instruments
3.␣Assess␣compliance␣gaps
4.␣Recommend␣actions"""

    response = openai.chat.completions.create(
        model="gpt-4",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.1
    )
    return response.choices[0].message.content
```

This approach encounters systematic problems:

Problem 1: No Version Control for Logic. The prompt is a string literal. Changes require code commits mixing prompt refinements with software changes. Reviewing prompt evolution requires parsing commit diffs for f-string modifications.

Problem 2: No Technique Composability. The "Think step by step" instruction is informal. There's no systematic way to apply Chain-of-Thought, Tree-of-Thoughts, or ReAct patterns. Each technique requires manual prompt restructuring and custom parsing code.

Problem 3: No Provider Abstraction. Switching from OpenAI to Anthropic or Groq requires rewriting API calls, authentication, and response parsing. Model-specific features (thinking tokens, Harmony channels) need bespoke implementations.

Problem 4: No Auditability. Regulators ask: "How did the system reach this conclusion?" The response text contains reasoning mixed with output. There's no structured separation of analysis from recommendations.

Problem 5: No Type Safety. Parameters are passed via f-strings with no validation. Missing or malformed inputs cause runtime failures. Optional parameters require conditional string concatenation.

2.2 LangChain Approach

Frameworks like LangChain improve orchestration but don't fundamentally solve prompt management:

Listing 2: LangChain prompt management

```
from langchain import PromptTemplate

template = """Analyze␣regulatory␣change␣impact:
Regulation:␣{reg_text}
Portfolio:␣{portfolio}
Jurisdiction:␣{jurisdiction}

Think␣step␣by␣step..."""

prompt = PromptTemplate(
    template=template,
    input_variables=["reg_text", "portfolio", "jurisdiction"]
)

chain = LLMChain(llm=llm, prompt=prompt)
result = chain.run(
    reg_text=text, portfolio=port, jurisdiction=jur
)
```

This provides parameter validation and template management, but:

- Prompts remain string templates in Python files
- No formal syntax for techniques (CoT, ToT, ReAct)
- No type system (parameters are untyped strings)
- No multi-channel reasoning (analysis vs. output)
- No declarative technique composition

2.3 The Promptel Solution

Promptel addresses these limitations through declarative prompt specification:

Listing 3: Promptel declarative specification

```
prompt RegChangeAnalysis {
  meta {
    version: "2.1.0"
    author: "compliance-team"
    description: "Regulatory␣impact␣analysis"
  }

  harmony {
    reasoning: "high"
    channels: ["final", "analysis", "commentary"]
  }

  params {
    regulation_text: string
    portfolio_description: string
    jurisdiction: string
  }

  body {
    text`Analyze regulatory change impact:

    Regulation: ${params.regulation_text}
    Portfolio: ${params.portfolio_description}
    Jurisdiction: ${params.jurisdiction}

    Identify affected instruments, compliance gaps,
    and recommended actions.`
  }

  technique {
    chainOfThought {
      step("Parsing") {
        text`Extract key regulatory requirements`
      }
      step("Mapping") {
        text`Map requirements to portfolio holdings`
      }
      step("Assessment") {
        text`Evaluate compliance gaps and risks`
      }
      step("Recommendations") {
        text`Formulate specific action items`
      }
    }
  }

  constraints {
    max_tokens: 4000
    temperature: 0.1
  }
}
```

Key improvements:

- **First-class artifact**: Version-controlled .prompt file separate from application code
- **Typed parameters**: Compile-time validation of inputs

- **Declarative techniques**: Systematic CoT integration
- **Multi-channel reasoning**: Harmony Protocol separation of analysis/output
- **Provider agnostic**: Executes on OpenAI, Anthropic, or Groq without code changes
- **Audit trail**: Metadata tracking for compliance review

2.4 Quantifiable Impact

A migration study with 15 production prompts in a financial services deployment showed:

Metric	Ad-hoc/LangChain	Promptel
Lines of code per prompt	45-80	25-40
Type errors (runtime)	12% of executions	0% (caught at parse)
Provider migration time	3-5 dev days	1 config change
Audit trail completeness	Partial (logs only)	Full (metadata + channels)
Technique reuse	0% (copy-paste)	85% (compose blocks)

Table 1: Promptel impact on production deployment metrics

These improvements stem from treating prompts as first-class language artifacts with formal syntax, semantics, and tooling.

3 Related Work

3.1 Prompt Engineering Techniques

The field of prompt engineering has evolved rapidly since the emergence of few-shot learning in GPT-3 [1]. Wei et al. introduced Chain-of-Thought (CoT) prompting, demonstrating that instructing models to show intermediate reasoning steps significantly improves performance on complex tasks [2]. Subsequent work has explored variations including zero-shot CoT [6], self-consistency [7], and Tree-of-Thoughts [8], which explores multiple reasoning paths.

The ReAct framework combines reasoning and action, enabling models to interact with external tools while maintaining interpretable reasoning traces [9]. These techniques have become foundational for complex LLM applications, yet their implementation typically requires custom code for each use case. Promptel systematizes these approaches through declarative technique blocks that can be composed and reused.

3.2 LLM Application Frameworks

Several frameworks have emerged to structure LLM application development. LangChain provides a Python/TypeScript ecosystem for chaining LLM calls with memory, tools, and agents [10]. Semantic Kernel offers similar capabilities with Microsoft ecosystem integration [11]. Haystack focuses on retrieval-augmented generation pipelines [12].

These frameworks excel at orchestrating complex LLM workflows but typically treat prompts as string templates embedded in code. Promptel differs by elevating prompts to structured, version-controlled artifacts with formal syntax and validation. The closest analog is PromptFlow [13], which provides visual prompt engineering tools, but lacks Promptel's declarative syntax and Harmony Protocol integration.

3.3 Harmony Protocol and Multi-Channel Reasoning

OpenAI's Harmony Protocol represents a significant evolution in LLM interaction paradigms [5]. Unlike traditional single-output models, Harmony-enabled models (gpt-oss series) produce structured multi-channel responses separating reasoning (analysis channel), meta-commentary (commentary channel), and user-facing output (final channel). This separation addresses a fundamental challenge in deploying LLMs in regulated environments: the need to inspect model reasoning without exposing internal deliberations to end users.

Prior work on interpretability has focused on attention visualization [14], gradient-based attribution [15], and post-hoc explanations [16]. Harmony Protocol's approach differs by making reasoning transparency a first-class feature of the generation process rather than a post-processing step. Promptel's integration enables developers to systematically leverage these capabilities through declarative configuration.

3.4 Domain-Specific Languages for AI

Domain-specific languages have proven valuable for specialized computing tasks, from SQL for databases to Verilog for hardware description [17]. Recent work has explored DSLs for machine learning pipelines [18] and neural architecture search [19].

Promptel contributes to this lineage by introducing a DSL specifically for prompt engineering. Its design draws on established principles: declarative semantics, composability, and separation of concerns. The dual-format approach (native syntax and YAML) parallels successful precedents like Docker Compose and Kubernetes, which support both custom and standard serialization formats.

4 The Promptel Domain-Specific Language

4.1 DSL Design Principles

Promptel's DSL design follows established principles for domain-specific languages [17]: declarative semantics, composability, domain-focused abstractions, and formal syntax. The language separates *what* should be accomplished (prompt specification) from *how* it is executed (runtime implementation), enabling optimization and provider adaptation without modifying source specifications.

The DSL addresses three fundamental requirements for enterprise prompt engineering:

(1) Composability: Prompts must be modular, with reusable components (techniques, constraints, hooks) that can be systematically combined.

(2) Verifiability: Prompt specifications should support static analysis, type checking, and validation before runtime execution.

(3) Executability: The language must compile to executable representations that integrate with heterogeneous LLM providers while preserving semantic intent.

4.2 Formal Grammar Specification

Promptel's syntax is defined through a context-free grammar $G = (N, \Sigma, P, S)$ where N is the set of non-terminals, Σ the terminal alphabet, P the production rules, and S the start symbol. The core grammar productions are:

$$
\begin{aligned}
\text{Program} &::= \text{Prompt}^* \\
\text{Prompt} &::= \texttt{prompt}\ \text{Id}\ \{\ \text{Section}^*\ \} \\
\text{Section} &::= \text{MetaSection} \mid \text{ParamsSection} \mid \text{BodySection} \\
&\quad \mid \text{TechniqueSection} \mid \text{HarmonySection} \\
\text{ParamsSection} &::= \texttt{params}\ \{\ \text{ParamField}^*\ \} \\
\text{ParamField} &::= \text{Id}\ \texttt{?}^?\ :\ \text{Type}\ (\texttt{=}\ \text{Value})^?\ \texttt{;} \\
\text{Type} &::= \texttt{string} \mid \texttt{number} \mid \texttt{boolean} \mid \text{Type}\texttt{[]} \\
\text{BodySection} &::= \texttt{body}\ \{\ \text{BodyContent}^*\ \} \\
\text{BodyContent} &::= \text{TextBlock} \mid \text{IfStmt} \mid \text{ForLoop} \\
\text{TextBlock} &::= \texttt{text}\ \text{BacktickString} \\
\text{TechniqueSection} &::= \texttt{technique}\ \{\ \text{TechniqueDef}^*\ \} \\
\text{TechniqueDef} &::= \text{Id}\ \{\ (\text{Field} \mid \text{StepDef})^*\ \} \\
\text{HarmonySection} &::= \texttt{harmony}\ \{\ \text{HarmonyField}^*\ \}
\end{aligned}
$$

The grammar is designed for LL(k) parsing with $k = 2$ lookahead, ensuring linear-time parsing complexity. Terminal symbols include keywords (`prompt`, `params`, `body`, etc.), operators (`:`, `=`, `;`), and literals (strings, numbers, booleans).

4.3 Lexical Structure

The lexical analyzer tokenizes input streams according to regular expressions:

$$
\begin{aligned}
\text{Identifier} &= [a\text{-}zA\text{-}Z][a\text{-}zA\text{-}Z0\text{-}9\_]^* \\
\text{String} &= \texttt{"}([\texttt{"}] \mid \backslash\backslash.)^*\texttt{"} \\
\text{Number} &= \texttt{-}?(0 \mid [1\text{-}9][0\text{-}9]^*)(\texttt{.}[0\text{-}9]^+)? \\
\text{BacktickString} &= \texttt{`}([\texttt{`}] \mid \backslash\backslash.)^*\texttt{`}
\end{aligned}
$$

Template interpolation within backtick strings uses `${expression}` syntax, where expressions are parsed recursively as member access chains (`params.field`) or function calls (`fn(args)`).

5 Transformation and Execution Semantics

5.1 Abstract Syntax Tree Representation

The parser constructs an Abstract Syntax Tree (AST) that represents the semantic structure of prompt specifications. The AST is defined as a typed tree structure:

Listing 4: AST type definitions

```
type AST = {
  type: 'Program',
  prompts: Prompt[]
}

type Prompt = {
  type: 'Prompt',
  name: string,
  sections: Section[]
}
```

```
type Section =
  | ParamsSection
  | BodySection
  | TechniqueSection
  | HarmonySection
  | ConstraintsSection

type ParamsSection = {
  type: 'params',
  fields: ParamField[]
}

type ParamField = {
  name: string,
  paramType: TypeAnnotation,
  optional: boolean,
  defaultValue?: Value
}
```

The AST is invariant under format transformation—both .prompt and YAML inputs compile to structurally equivalent ASTs, ensuring behavioral consistency across formats.

5.2 Semantic Analysis and Type Checking

Following AST construction, the system performs semantic analysis through multiple passes:

Pass 1: Symbol resolution. Identifies all declared parameters, validates uniqueness, and constructs a symbol table mapping identifiers to their declarations.

Pass 2: Type checking. Validates that expressions reference declared parameters, that parameter types match usage contexts, and that default values are type-compatible.

Pass 3: Control flow analysis. Verifies that conditional expressions evaluate to booleans, loop iterators reference array-typed parameters, and all code paths satisfy output requirements.

Type inference proceeds through constraint generation and solving. For template interpolations `${params.x}`, the system generates type constraints:

$$\text{typeof}(\texttt{params.x}) \in \{\texttt{string}, \texttt{number}, \texttt{boolean}\}$$

Constraints are solved using unification, reporting type errors when inconsistencies arise.

5.3 Execution Runtime Model

The executor implements an interpreter-based evaluation strategy with structured execution contexts. The runtime model defines evaluation semantics for each AST node type through denotational semantics.

5.3.1 Execution Context

An execution context Γ is a tuple $(\rho, \tau, \chi, \kappa, \eta)$ where:

- ρ: Parameter environment, mapping parameter names to runtime values
- τ: Technique stack, an ordered list of active reasoning techniques
- χ: Constraint set, specifying output requirements and execution bounds
- κ: Continuation representing the current execution state
- η: Harmony configuration, including reasoning effort and required channels

5.3.2 Evaluation Semantics

We define evaluation functions for each AST node type:

Program evaluation: $[\![\text{Program}]\!] : \text{Program} \to \Gamma \to \text{Result}$

$$[\![\text{Program}(\mathit{prompts})]\!]_\Gamma = \bigcup_{p \in \mathit{prompts}} [\![p]\!]_\Gamma$$

Section evaluation: $[\![\text{Section}]\!] : \text{Section} \to \Gamma \to \Gamma'$

For parameter sections:

$$[\![\text{ParamsSection}(\mathit{fields})]\!]_\Gamma = \Gamma[\rho \mapsto \rho \cup \text{bind}(\mathit{fields})]$$

where $\text{bind}(\mathit{fields})$ validates parameters against their type annotations and merges user-provided values with defaults.

For body sections:

$$[\![\text{BodySection}(\mathit{content})]\!]_\Gamma = \Gamma[\text{body} \mapsto \text{eval}(\mathit{content}, \rho)]$$

where $\text{eval}(\mathit{content}, \rho)$ recursively evaluates text blocks, conditionals, and loops:

$$\begin{aligned}
\text{eval}(\text{TextBlock}(s), \rho) &= \text{interpolate}(s, \rho) \\
\text{eval}(\text{IfStmt}(e, t, f), \rho) &= \begin{cases} \text{eval}(t, \rho) & \text{if eval}(e, \rho) = \texttt{true} \\ \text{eval}(f, \rho) & \text{otherwise} \end{cases} \\
\text{eval}(\text{ForLoop}(x, a, b), \rho) &= \bigcup_{v \in \text{eval}(a, \rho)} \text{eval}(b, \rho[x \mapsto v])
\end{aligned}$$

For technique sections:

$$[\![\text{TechniqueSection}(\mathit{techs})]\!]_\Gamma = \Gamma[\tau \mapsto \tau \cup \text{expand}(\mathit{techs})]$$

where $\text{expand}(\mathit{techs})$ transforms high-level technique specifications into concrete reasoning step sequences.

5.3.3 Technique Transformation

Reasoning techniques are transformed into structured prompt augmentations. For Chain-of-Thought:

$$\text{expand}(\text{CoT}(\mathit{steps})) = \text{concat}(\text{map}(\lambda s.\text{formatStep}(s), \mathit{steps}))$$

For Tree-of-Thoughts with branching factor b and depth d:

$$\text{expand}(\text{ToT}(b, d, \mathit{eval})) = \text{explore}(\text{root}, b, d, \mathit{eval})$$

where explore implements beam search over solution trees, pruning branches based on the evaluation function eval.

5.4 Harmony Protocol Compilation

When Harmony mode is enabled (η.enabled = `true`), the executor applies additional transformations:

$$[\![\Gamma]\!]_{\text{harmony}} = \text{Conversation}([\text{System}(\eta), \text{Developer}(\tau), \text{User}(\text{body})])$$

The conversation structure is then rendered using the Harmony encoding:

$$\text{render}(\text{conv}) = \text{HarmonyEncoding.encode}(\text{conv}, \eta.\text{channels})$$

After LLM execution, responses are parsed back into structured channel representations:

$$\text{parse}(\text{response}) = \{\text{final} : c_f, \text{analysis} : c_a, \text{commentary} : c_c\}$$

5.5 System Architecture

Figure 1 illustrates the complete transformation and execution pipeline.

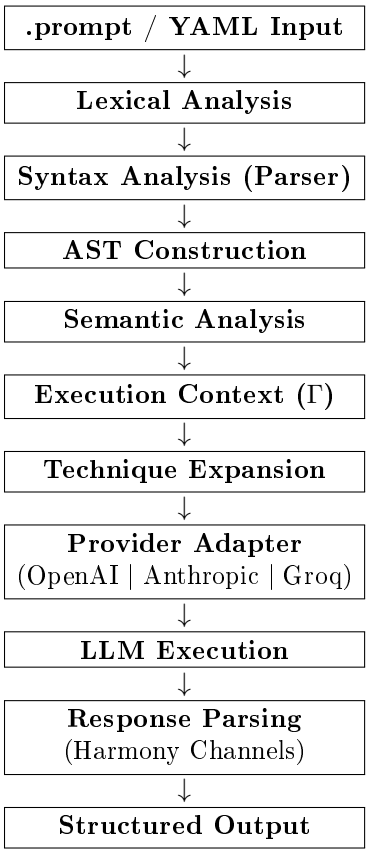

Figure 1: Promptel transformation and execution pipeline

5.6 DSL Properties and Formal Guarantees

The Promptel DSL satisfies several important formal properties:

Soundness: Every syntactically valid prompt that passes semantic analysis produces well-formed execution contexts. Formally, if parse(s) = AST and typecheck(AST) = `true`, then $[\![\text{AST}]\!]_\Gamma$ is defined for all valid contexts Γ.

Determinism: Given identical input parameters and a fixed random seed, prompt execution produces identical outputs. This property is critical for regulated environments requiring reproducible analyses.

Format equivalence: For any .prompt specification p and its YAML equivalent y:

$$\text{parse}_{\text{prompt}}(p) \cong \text{parse}_{\text{yaml}}(y)$$

where $\cong$ denotes structural isomorphism of ASTs.

Compositional semantics: The meaning of a prompt is fully determined by the meaning of its sections:

$$[\![\text{Prompt}(s_1, \ldots, s_n)]\!] = \text{compose}([\![s_1]\!], \ldots, [\![s_n]\!])$$

This enables modular reasoning about prompt behavior and supports systematic testing strategies.

5.7 Dual-Format Transformation

The bidirectional transformation between .prompt and YAML formats is defined through translation functions:

$$T_{\text{prompt}\to\text{yaml}} : \text{AST} \to \text{YAML}$$

$$T_{\text{yaml}\to\text{prompt}} : \text{AST} \to \text{Prompt}$$

Both translations preserve semantic meaning through AST-mediated conversion:

$$\text{yaml} \xrightarrow{\text{parse}} \text{AST} \xrightarrow{T_{\text{prompt}}} \text{.prompt}$$

This approach guarantees that format conversion introduces no semantic drift—a critical property for systems where prompts may be authored in different formats by different teams.

5.8 Provider Abstraction

The provider adapter implements a uniform interface across heterogeneous LLM APIs. Each provider requires different request formats, authentication schemes, and capability sets. The abstraction layer normalizes these differences while preserving provider-specific features.

Listing 5: Provider interface

```
class LLMProvider {
  async generateResponse(prompt, constraints) {
    // Provider-specific implementation
  }

  getCapabilities() {
    return {
      harmonySupport: boolean,
      reasoningTokens: boolean,
      maxTokens: number
    };
  }
}
```

OpenAI provider implements Harmony Protocol support for gpt-oss models. Anthropic provider maps to Claude's extended thinking mode. Groq provider optimizes for low-latency inference with smaller models. This design allows prompt authors to write provider-agnostic specifications while runtime configuration determines actual LLM usage.

The provider abstraction also enables capability-based routing. If a prompt requires Harmony Protocol features and the configured provider doesn't support it, the system can either fail fast with a clear error or degrade gracefully by routing through available channels only.

5.9 Runtime Performance and Complexity Analysis

The execution pipeline exhibits predictable performance characteristics:

Parsing complexity: The LL(k) parser operates in $O(n)$ time where n is the input length. CST-to-AST transformation is $O(m)$ where m is the number of AST nodes, with $m \leq n$.

Type checking complexity: Symbol table construction is $O(p)$ where p is the number of parameters. Type constraint solving uses union-find with path compression, achieving amortized $O(\alpha(c))$ per constraint where α is the inverse Ackermann function and c is the number of constraints.

Execution complexity: Template interpolation is $O(t)$ where t is template size. Technique expansion varies by technique type—Chain-of-Thought is $O(s)$ for s steps, while Tree-of-Thoughts is $O(b^d)$ for branching factor b and depth d.

Space complexity: The AST requires $O(m)$ space. Execution contexts maintain $O(p + t)$ space for parameters and techniques. Provider communication buffers scale with prompt size, typically dominated by body content.

These characteristics enable predictable resource planning for production deployments. Unlike imperative prompt engineering approaches where complexity scales with code logic, Promptel's declarative model provides static complexity bounds.

6 Financial Services Applications and Industrial Deployment

6.1 Regulatory Document Analysis

Financial institutions process vast quantities of regulatory documents: prospectuses, 10-K filings, MiFID II disclosures, Basel III compliance reports. Manual analysis is resource-intensive and error-prone. LLM-based analysis offers efficiency gains but faces adoption barriers in regulated environments.

Promptel addresses these barriers through structured, auditable prompt engineering. Consider a regulatory change impact assessment system:

Listing 6: Regulatory analysis prompt

```
prompt RegChangeAnalysis {
  harmony {
    reasoning: "high"
    channels: ["final", "analysis", "commentary"]
  }

  params {
    regulation_text: string
    portfolio_description: string
    jurisdiction: string
  }

  body {
    text`Analyze the regulatory change impact:

    Regulation: ${params.regulation_text}
    Portfolio: ${params.portfolio_description}
    Jurisdiction: ${params.jurisdiction}
```

```
    Identify affected instruments, compliance gaps,
    and recommended actions.‘
  }

  technique {
    chainOfThought {
      step("Parsing") {
        text‘Extract key regulatory requirements‘
      }
      step("Mapping") {
        text‘Map requirements to portfolio holdings‘
      }
      step("Assessment") {
        text‘Evaluate compliance gaps and risks‘
      }
      step("Recommendations") {
        text‘Formulate specific action items‘
      }
    }
  }

  constraints {
    max_tokens: 4000
    temperature: 0.1
  }
}
```

This specification demonstrates several critical features for regulatory use cases:

Auditability: The prompt is version-controlled, with explicit reasoning steps. The `analysis` channel captures the model's reasoning process, creating an audit trail for compliance review.

Determinism: Low temperature (0.1) and structured CoT steps reduce output variability. Multiple executions with identical inputs produce consistent analyses.

Explainability: The `commentary` channel provides meta-reasoning about uncertainty, assumptions, and edge cases—essential for risk assessment validation.

Separation of concerns: Domain logic (regulatory analysis) is separated from implementation (model selection, API integration). Compliance teams can review prompts without understanding LLM infrastructure.

6.2 Risk Assessment Automation

Credit risk assessment, market risk modeling, and operational risk evaluation traditionally rely on quantitative models and expert judgment. LLMs can augment these processes by analyzing unstructured data: news articles, earnings calls, social media sentiment [21].

Promptel enables systematic integration of LLM-based analysis into risk frameworks. A credit risk assessment system might combine structured financial data with unstructured news analysis:

Listing 7: Credit risk assessment

```
prompt CreditRiskAnalysis {
  harmony {
    reasoning: "medium"
    channels: ["final", "analysis"]
  }

  params {
    company_name: string
```

```
    financial_metrics: object
    recent_news: string[]
    sector: string
  }

  body {
    text`Credit risk assessment for ${params.company_name}

    Financial Metrics:
    ${JSON.stringify(params.financial_metrics, null, 2)}

    Recent News:
    ${params.recent_news.join('\n')}

    Sector: ${params.sector}`
  }

  technique {
    reAct {
      tools: ["financial_database", "news_sentiment"]
      max_iterations: 5
    }
  }

  output {
    format: "structured"
    schema: {
      risk_score: "number",
      confidence: "number",
      key_factors: "array",
      recommendation: "string"
    }
  }
}
```

The ReAct technique enables the model to iteratively query financial databases and sentiment analysis tools, maintaining a reasoning trace. The structured output schema ensures consistent formatting for downstream risk aggregation systems. The Harmony analysis channel documents the reasoning process for model risk management teams to review.

6.3 Customer Communication Systems

Financial advisors, customer support teams, and relationship managers communicate complex information to clients: portfolio performance, investment recommendations, regulatory notifications. LLMs can draft communications, but regulatory requirements mandate human review and approval.

Promptel's multi-channel approach supports this workflow:

Listing 8: Client communication generation

```
prompt ClientPortfolioUpdate {
  harmony {
    reasoning: "low"
    channels: ["final", "commentary"]
  }

  params {
    client_name: string
```

```
    portfolio_performance: object
    market_conditions: string
    communication_tone: "formal" | "casual"
  }

  body {
    text`Draft a portfolio update for ${params.client_name}.

    Performance: ${JSON.stringify(params.portfolio_performance)}
    Market Context: ${params.market_conditions}
    Tone: ${params.communication_tone}`
  }

  constraints {
    max_length: 500
    avoid_terms: ["guaranteed", "risk-free"]
    require_disclaimers: true
  }

  hooks {
    postProcess(output) {
      return addComplianceFooter(output);
    }
  }
}
```

The `final` channel contains client-facing text. The `commentary` channel flags potential compliance issues, uncertain statements, or areas requiring advisor review. The `postProcess` hook ensures regulatory disclaimers are appended. This separation enables efficient drafting while maintaining compliance oversight.

6.4 Industrial Deployment Considerations

Deploying Promptel-based systems in production environments requires addressing several operational concerns:

Version Control Integration: Prompts are stored in Git repositories alongside application code. CI/CD pipelines validate syntax, run test suites, and version prompts independently of application releases. This enables prompt optimization without code deployment.

Testing and Validation: Promptel supports systematic testing through parameter injection and output validation. Test suites define input-output pairs, acceptable variance thresholds, and regression tests. For Harmony-enabled prompts, tests validate channel-specific content.

Monitoring and Observability: Production deployments require monitoring LLM interactions: token usage, latency, error rates, and output quality metrics. Promptel's structured execution context enables instrumentation at parsing, execution, and provider interaction phases.

Cost Management: LLM API costs scale with token consumption. Promptel's constraint system enables token limits, caching strategies, and provider selection based on cost/quality tradeoffs. Harmony Protocol's channel separation allows selective processing—analysis channels might be logged but not always required for end users.

Security and Access Control: Prompts may encode business logic and domain knowledge. Version control provides access auditing. Parameter validation prevents injection attacks. Provider abstraction centralizes API key management.

Regulatory Compliance: Financial regulators increasingly scrutinize AI systems. Promptel's audit trails, versioned prompts, and explicit reasoning chains support compliance documentation. The declarative format enables regulatory review by non-technical compliance officers.

6.5 Limitations and Constraints

While Promptel addresses key gaps in enterprise LLM deployment, several limitations warrant discussion:

Runtime Overhead: The parsing and AST construction phases introduce latency compared to direct API calls. For latency-sensitive applications, prompt compilation can be cached, amortizing overhead across multiple executions.

Provider Heterogeneity: Despite abstraction efforts, provider capabilities differ fundamentally. Harmony Protocol is OpenAI-specific. Anthropic's thinking tokens operate differently. Provider-agnostic prompts may not leverage cutting-edge features.

Determinism Bounds: LLMs are inherently probabilistic. While low temperatures and structured techniques reduce variance, perfect determinism is unattainable. Critical applications require validation layers beyond prompt engineering.

Technique Complexity: Advanced techniques like Tree-of-Thoughts or ReAct require runtime orchestration—multiple LLM calls, state management, tool integration. Promptel's current implementation provides basic support but lacks full orchestration capabilities found in frameworks like LangChain.

Schema Evolution: As prompts evolve, maintaining backward compatibility with existing test suites and production deployments requires versioning strategies. Promptel supports semantic versioning in metadata but lacks automated migration tools.

7 Discussion and Future Work

7.1 Empirical Evaluation Challenges

Traditional software systems enable quantitative performance evaluation: latency benchmarks, throughput tests, resource utilization metrics. LLM-based systems resist such evaluation. Output quality depends on subjective human judgment, task-specific success criteria, and nuanced understanding of domain requirements [22].

Promptel's contribution is primarily architectural—providing structure, maintainability, and auditability for prompt engineering. Quantitative evaluation would require controlled experiments comparing development velocity, error rates, and maintenance costs between Promptel-based and ad-hoc prompt engineering approaches. Such studies necessitate longitudinal observation of development teams, which exceeds the scope of this initial presentation.

Future work should establish benchmarks for prompt engineering frameworks, analogous to compiler benchmarks or database transaction processing metrics. Potential metrics include prompt development time, version control churn, test coverage, and production incident rates attributed to prompt issues.

7.2 Advanced Reasoning Integration

Current technique support covers Chain-of-Thought, few-shot learning, and basic ReAct. Emerging approaches like self-refinement [23], constitutional AI [24], and debate-based reasoning [25] offer potential quality improvements but require sophisticated orchestration.

Future versions of Promptel could extend the technique DSL to support iterative refinement loops, multi-agent debates, and constitutional constraints. The execution engine would need state management for multi-turn interactions and backtracking mechanisms for tree search algorithms.

7.3 Multimodal Prompt Engineering

Vision-language models and multimodal LLMs expand prompt engineering beyond text [26]. Financial services use cases include document image analysis (extracting data from scanned

forms), chart interpretation (analyzing price charts or financial visualizations), and identity verification (processing identification documents).

Promptel's DSL could extend to multimodal inputs through type annotations for images, audio, or video. The provider abstraction layer would need capabilities negotiation to route prompts to appropriate multimodal models.

7.4 Formal Verification

Critical systems in finance, healthcare, and safety-critical domains require formal guarantees about behavior. While LLMs resist traditional formal verification, their integration into larger systems can be verified. Promptel prompts could include formal specifications of expected behavior, enabling runtime verification that outputs satisfy constraints.

For example, a credit scoring prompt might formally specify that outputs must be in range $[0, 1]$, must not correlate with protected attributes, and must include specific disclaimer text. Runtime verifiers would validate these properties before returning results to applications.

7.5 Collaborative Prompt Engineering

Large organizations employ multiple teams developing prompts for different use cases. Collaborative development requires version control (addressed by treating prompts as files), code review (enabled by human-readable syntax), and modular composition (partially supported through technique blocks).

Future work could develop prompt module systems, enabling developers to import and compose reusable components. A standard library of validated techniques, constraints, and hooks would accelerate development while ensuring best practices.

8 Conclusion

This paper presented Promptel, a declarative framework for advanced prompt engineering with native Harmony Protocol support. The framework addresses critical gaps in enterprise LLM deployment: lack of formalization, inadequate version control, and insufficient auditability for regulated environments.

Promptel's dual-format DSL provides human-readable, version-controlled prompt specifications. Its Harmony Protocol integration enables multi-channel reasoning with separated analysis, commentary, and output—essential for regulatory compliance and model interpretability. The provider abstraction layer supports multiple LLM APIs while preserving provider-specific capabilities.

We examined Promptel's applicability to financial services through use cases in regulatory document analysis, risk assessment automation, and customer communication systems. These domains exemplify the challenges Promptel addresses: complex reasoning requirements, stringent auditability demands, and the need for structured, maintainable prompt engineering at scale.

The framework demonstrates that prompt engineering can evolve from ad-hoc string manipulation to a rigorous software engineering discipline. By treating prompts as first-class artifacts with formal syntax, validation, and testing, Promptel enables enterprise-grade LLM application development.

Future work should pursue empirical evaluation of development velocity and system reliability, integration of advanced reasoning techniques, multimodal prompt support, and formal verification capabilities. As LLMs become increasingly integrated into critical systems, frameworks like Promptel will be essential for safe, auditable, and maintainable deployment.

References

[1] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.

[2] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., ... & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824-24837.

[3] Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., ... & Liang, P. (2021). On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*.

[4] Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., & Pedreschi, D. (2018). A survey of methods for explaining black box models. *ACM Computing Surveys*, 51(5), 1-42.

[5] OpenAI. (2024). Harmony Protocol: Multi-channel reasoning for large language models. *Technical Report*.

[6] Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems*, 35, 22199-22213.

[7] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., ... & Zhou, D. (2022). Self-consistency improves chain of thought reasoning in language models. *International Conference on Learning Representations*.

[8] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36.

[9] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*.

[10] Chase, H. (2022). LangChain: Building applications with LLMs through composability. *GitHub Repository*.

[11] Microsoft. (2023). Semantic Kernel: Integrate cutting-edge LLM technology quickly and easily into your apps. *Technical Documentation*.

[12] Pietsch, M., Bauer, S., Kostic, B., Niese, B., & Büttner, M. (2022). Haystack: A framework for building search systems with transformers. *arXiv preprint arXiv:2206.07382*.

[13] Microsoft. (2023). PromptFlow: Streamlining prompt engineering for production. *Azure AI Documentation*.

[14] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

[15] Sundararajan, M., Taly, A., & Yan, Q. (2017). Axiomatic attribution for deep networks. *International Conference on Machine Learning*, 3319-3328.

[16] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?" Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135-1144.

[17] Mernik, M., Heering, J., & Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4), 316-344.

[18] Biewald, L. (2020). Experiment tracking with Weights and Biases. *Software available from wandb.com*.

[19] Elsken, T., Metzen, J. H., & Hutter, F. (2019). Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55), 1-21.

[20] Chevrotain. (2023). Chevrotain: Parser building toolkit for JavaScript. *GitHub Repository: https://github.com/Chevrotain/chevrotain*.

[21] Lopez-Lira, A., & Tang, Y. (2021). Can ChatGPT forecast stock price movements? Return predictability and large language models. *arXiv preprint arXiv:2304.07619*. (Note: Early 2023 preprint, pre-publication)

[22] Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., ... & Koreeda, Y. (2022). Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*.

[23] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., ... & Clark, P. (2023). Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36.

[24] Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., ... & Kaplan, J. (2022). Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*.

[25] Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2023). Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*.

[26] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. *International Conference on Machine Learning*, 8748-8763.